

Where do videos go to die?

Speaker script for the research update, October 5, 2026

Wun Ting Chan · 24 slides · about 24 minutes

The arc in one paragraph. We open with a striking picture: long AI videos all die the same death. We turn that into a question anyone can follow: where do long videos go, how fast, and which design choice decides? Then we explain how we got here. A year of trying to fix the decay failed in an instructive way, and that failure is why we switched to measuring it. The rest of the talk builds the case one piece at a time, each slide answering the doubt the previous one raises. Is this real? (Real video doesn't do it.) Is it one phenomenon? (No, three distinct fates.) Can we measure it? (Yes: a shared attracting regime, a one-minute half-life.) Is it fixed by the weights? (No: a memory setting changes the regime.) Can simple statistics predict collapse? (Not the ones we tried.) We close with what is new, stated carefully against prior measurement work, and what comes next.

[Bracketed grey text is stage direction: where to point, when to pause.]

Slide 1 Title



Good afternoon. The title is a little dramatic on purpose: where do videos go to die?

Over the next twenty minutes I want to convince you of three things. First, long AI-generated videos don't just slowly get worse: different generators settle into qualitatively different long-run regimes. Second, we can measure those regimes by treating the generator as a prompt-conditioned stochastic dynamical system, using tools borrowed from physics, applied operationally. Third, which regime you get can be changed by a memory setting, with the weights held fixed.

This project began life as a test-time adaptation project. Part of this talk is the story of why it became something else.

Slide 2 AI can now generate video that never ends

The question
0000

How far can we go?
00

Algorithm
00

Results
000000000000000000

Next
0

AI can now generate video that never ends

► New *streaming* video models write a video one short chunk at a time, each chunk looking at the last few.

► In principle they can run forever: games, live worlds, endless scenes.

► We ran three of them for **3 minutes** on 8 very different prompts.

Tokyo street

Woolly mammoths

Space-man trailer

Big Sur waves

Fluffy monster

Paper coral reef

Crowned pigeon

Pirate ships in coffee

The first frame of each of the 8 prompts (Self Forcing).

What happens to a video if you just keep going?

Where do videos go to die? Win-Ting Chan 2

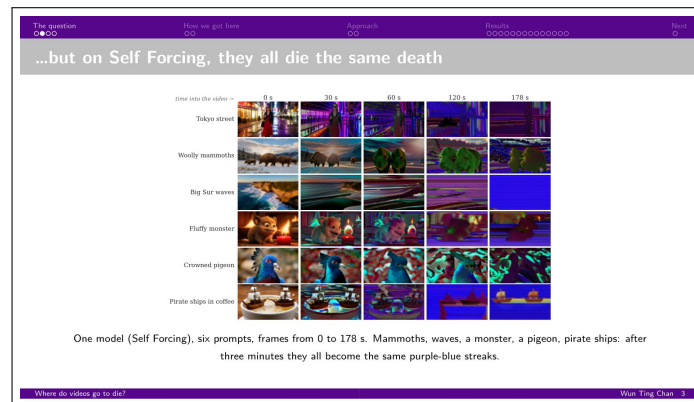
Point of this slide: set up the promise before showing the failure.

Here's the setting. A new family of video models, called streaming or autoregressive models, doesn't generate a whole clip at once. It writes a few frames, looks back at what it just wrote, and writes the next few. In principle that loop never has to stop. This is the technology behind live interactive worlds and game-like video.

[point at the grid] These are the opening frames from one of those models, given eight very different prompts: a woman on a Tokyo street, woolly mammoths, a sci-fi trailer, waves at Big Sur, a fluffy cartoon monster, a paper coral reef, a crowned pigeon, and pirate ships fighting in a cup of coffee. The openings look great. They're varied, detailed and on-prompt.

So the obvious experiment is: what happens if we just keep going? We let three of these models run for three full minutes, roughly 2,900 frames each.

Slide 3 ...but on Self Forcing, they all die the same death



Point of this slide: the hook. Let the image do the work.

[pause, let people look for a few seconds]

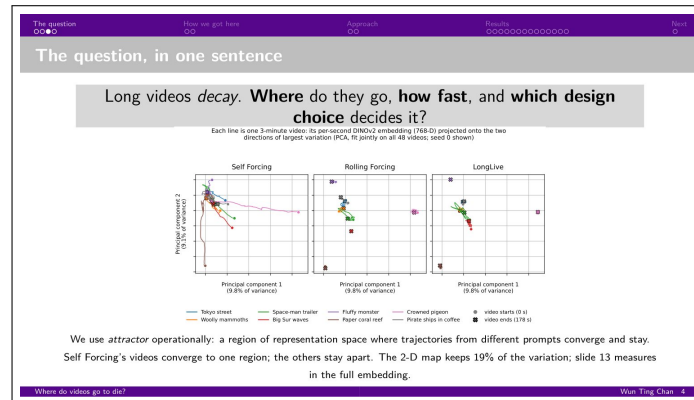
Each row is one prompt. Time runs left to right, from the first frame to three minutes. Read across any row and you watch a video decay. Now read down the last column.

The Tokyo street, the mammoths, the coastline, the monster, the pigeon, the pirate ships: by 178 seconds they've all turned into the same thing, purple and blue horizontal streaks. Six unrelated prompts arrive at one destination.

That's what I want you to remember. The decay isn't random noise piling up. It goes somewhere specific. For this model, everything goes to the same place.

Notice also that it isn't sudden. At 30 seconds you can already see the colour shift into neon purple. By 60 seconds the textures are breaking up. By 120 the content is mostly gone.

Slide 4 The question, in one sentence



Point of this slide: give the audience the mental model (marble and valley) before any technical vocabulary.

So here is the research question in one sentence. Long videos decay: where do they go, how fast, and which design choice decides it?

The picture to keep in mind is a marble on a hilly landscape. Wherever you drop it, it rolls downhill and settles in a valley. Physicists call that valley an attractor. We use the word operationally: a region of representation space where trajectories from different prompts converge and stay. We aren't claiming a proof about the model's full internal state.

[point at the three panels] This plot is our first evidence that the analogy is more than a metaphor. Its job is to show, in one picture, that the three models have differently shaped landscapes. Here is how to read it. For every frame of a video, an image encoder called DINOv2 gives a list of 768 numbers describing what's in the frame. We average those once per second, so a three-minute video becomes a path through that 768-dimensional space. To draw it, we find the two directions along which all the frames, from all 48 videos, vary the most. Those are the two principal components on the axes, and we project each path onto them. So each coloured line is the frame-by-frame trajectory of one generated video: the dot is its first frame, the cross is its last, and the colour tells you the prompt.

On the left, Self Forcing: the lines start all over the map and converge into one corner. That's a single valley everyone falls into. In the middle and on the right, Rolling Forcing and LongLive: each video barely moves from where it started. Every prompt has its own little valley.

One honest caveat, which is also in the caption: squashing 768 dimensions into two keeps only about 19% of the variation, so this map is a sketch. The actual claim rests on distances measured in the full space, which you'll see on slide 13.

Slide 5 Why this matters

Why this matters

A common problem, with very different outcomes

Streaming video, game engines and world models generate one step at a time from their own output. Long rollouts often accumulate drift, and recent designs (sinks, long-video tuning) reduce it a lot. In our runs, how a video ends depends strongly on the model and its inference settings.

Measured as symptoms, not as dynamics

Benchmarks and recent papers already quantify degradation: quality and consistency over long videos (VBench-Long), colour and motion drift, spectral drift, representation collapse, sink collapse. What we have not found is a characterization of the *dynamics*: where trajectories converge, how fast the prompt is lost, how perturbations grow, which design change alters the end state.

What we provide

- ▶ A map of each model's **end states**
- ▶ A **memory half-life** for the prompt
- ▶ **Perturbation growth**: does a tiny nudge change the story?
- ▶ **Regime-changing settings**: one memory setting changes the long-run regime
- ▶ Tools anyone can run on a new model

Images do videos go to die? Wen-Ting Chen 5

Point of this slide: place the work precisely: a common problem with different outcomes, already measured as symptoms, not yet as dynamics.

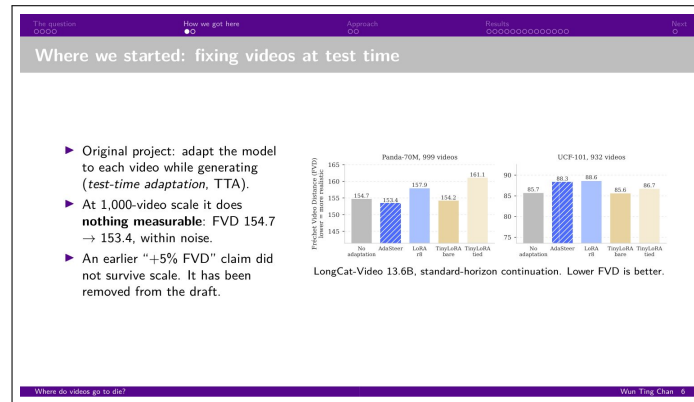
Why should anyone care beyond pretty failure pictures?

Everyone building streaming video, generative game engines or world models faces the same structural risk: the model feeds on its own output, so small errors can compound. Long rollouts often accumulate drift, and much of the recent literature is about reducing it, with anchor frames called sinks or extra training on long videos. As you'll see in a moment, it works to a large degree. Two of our three models hold up well for three minutes. So the honest framing isn't that every model collapses. It's that how a long video ends depends strongly on the model and how you run it, and we don't have a good way to describe that dependence.

That brings me to the second box. This problem is already measured, quite a lot: long-video benchmarks like VBench-Long, and recent papers that quantify colour drift, motion drift, spectral drift, representation collapse and sink collapse. What those papers measure are symptoms, one quality score at a time. What we couldn't find is a characterization of the *dynamics*: where trajectories converge, how fast the prompt is lost, how a small perturbation grows, and which design change moves the end state.

That's our contribution, on the right: a map of each model's end states, a memory half-life for the prompt, a perturbation-growth test, the finding that one memory setting can change the long-run regime, and tools anyone can run on a new model.

Slide 6 Where we started: fixing videos at test time



Point of this slide: be honest about the starting point; the null result is what motivates the pivot.

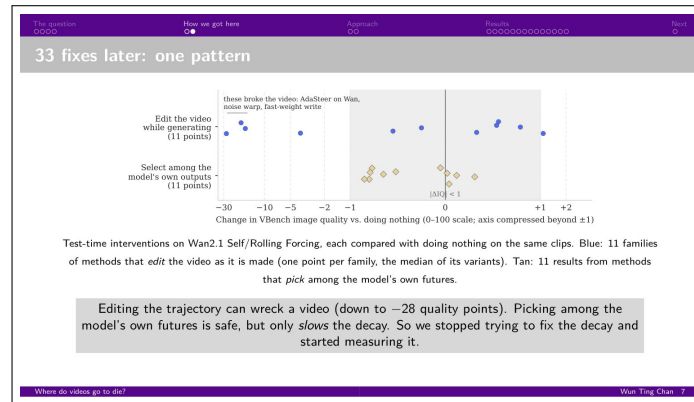
Some of you know this project under its old name, test-time adaptation. The idea was intuitive: while you generate a video, briefly fine-tune the model on that video so it stays faithful to it.

[point at the bars] This chart is the decisive experiment, and its job is to show that at scale the effect is zero. The metric is FVD, a standard measure of how realistic a set of videos looks; lower is better. Grey is doing nothing. The hatched blue bar is our method, AdaSteer. The others are standard adaptation baselines. On the left, Panda-70M with 999 videos; on the right, UCF-101 with 932.

Look at how flat it is. On Panda, doing nothing scores 154.7 and our method 153.4, a difference well inside the noise. On UCF, our method is slightly worse. The heavier baselines move things around but don’t improve anything. An earlier “five percent improvement” from smaller runs did not survive scaling up, and we’ve taken it out of the draft.

That’s a real result, but it’s a negative one. And it pushed us to ask a different question.

Slide 7 33 fixes later: one pattern



Point of this slide: the pivot. Turn a pile of failures into one principle.

We didn't stop at one method. Over the summer we tried many families of test-time fixes on the streaming models. This chart puts them on the same ruler, with the same number of points on each side so neither looks more crowded.

The purpose of this plot is to show a pattern across all those attempts, not to judge any single one. The horizontal axis is the change in image quality compared with doing nothing; zero is the vertical line. The axis is stretched so you can see small and catastrophic changes at once. The grey band is plus or minus one point, effectively no change.

Two kinds of method. The 11 blue dots on the top row are families of methods that edit the video as it's being made: they change the noise, the weights or the memory. Each dot is one family, at the median of the variants we tried. The 11 tan diamonds on the bottom row select: they let the model propose several futures and keep the best one.

The edits are spread across the whole axis. Some help a little, and some destroy the video: the three on the far left, AdaSteer on Wan, noise warp and fast-weight write, lost 17 to 28 quality points. The selection methods all sit inside the grey band. They're safe, but in other experiments they only slow the decay; they never stop it.

That was the moment of the pivot. If every fix either breaks the video or only delays the inevitable, maybe the decay itself is the interesting object. So we stopped trying to fix it and started measuring it.

Slide 8 Setup

Setup

Models (all on Wan2.1, 1.3B)

- **Self Forcing**: rolling window, no anchor
- **Rolling Forcing**: keeps the first chunk as an anchor ("sink")
- **LongLive**: sink plus long-video tuning

Measurements, every frame

- **Content**: DINOv2 embedding (what is in the frame)
- **Look**: brightness, contrast, saturation, colourfulness, sharpness, spectral slope
- **Motion**: frame difference, optical flow

Rollouts

8 MovieGen prompts $\times$ 2 seeds $\times$ 3 models = 48 videos of 3 minutes (2,877 frames each). Plus 128 real 30s clips as a control. About 8 H200-hours so far.

A prompt-conditioned stochastic dynamical system

$$X_{t+1} = F_{\theta, \lambda}(X_t; c, \xi_t)$$

X_t : state (recent latent frames and their memory cache)
 $F_{\theta, \lambda}$: the generator, weights θ , inference settings λ
 (sink, window) - c : prompt - ξ_t : generation noise. We ask how the distribution of X_t given c evolves.

Point of this slide: just enough detail to trust the results, and the formal framing everything else rests on.

A quick word on the setup, so you can trust what comes next.

Three models, all built on the same 1.3-billion-parameter Wan2.1 video backbone, so differences between them come from design, not size. Self Forcing uses a rolling window and no anchor. Rolling Forcing keeps the very first chunk around as a permanent anchor, which the field calls a “sink”. LongLive also has a sink, plus extra training on long videos.

We ran 8 prompts, 2 random seeds each, 3 models: 48 videos of three minutes. As a control, we also have 128 real video clips. So far this has cost about 8 GPU-hours.

On every frame we record what’s in it (an embedding from DINOv2, a standard image encoder), how it looks (brightness, saturation, sharpness and so on), and how much it moves (optical flow).

[bottom-right box] The key reframing is the equation. We treat the generator as a prompt-conditioned stochastic dynamical system. The state X_t is what the model carries forward: its recent latent frames and their memory cache. The transition rule F is the generator itself, with weights θ and inference settings λ such as sink size and attention window. c is the prompt, and ξ_t is the generation noise. Asking where long videos go becomes asking how the distribution of X_t , given the prompt, evolves over time.

Slide 9 Toolkit: questions about the dynamics, and how we measure them

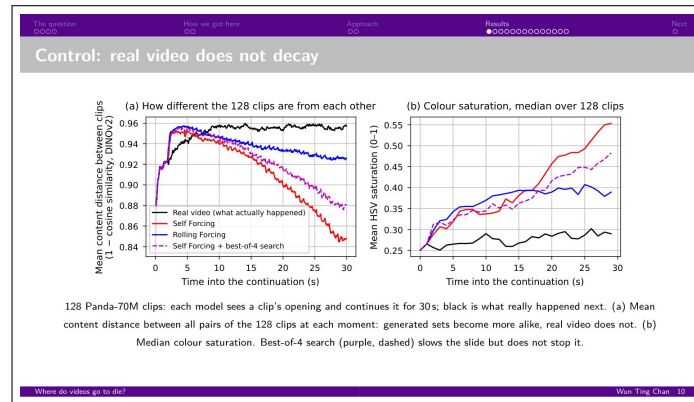
Toolkit: questions about the dynamics, and how we measure them		
Concept	Question	Measurement
Attractor	Where do trajectories concentrate?	Cross-prompt distances; can the prompt still be recovered from the frame?
Relaxation	How fast is conditioning information lost?	Exponential fit to that distance; prompt <i>half-life</i>
Perturbation growth	How does a small local change propagate?	Twin rollouts, one noise block nudged; finite-time growth rate
Regime transition	Which settings change long-run behaviour?	Same weights; change sink size and attention window
Recurrence	Does temporal ordering carry structure?	Recurrence analysis vs. phase-randomized surrogates
Predictability	Can failure be anticipated?	Pre-collapse variance and autocorrelation statistics
Physics terms are used operationally and measured in representation space; we do not claim proofs about the model's full internal state.		

Point of this slide: a roadmap for the results; each row is a slide that follows. Terms are operational.

This table is the roadmap for the rest of the talk. Each row takes a concept from dynamical systems, states it as a plain question, and names what we actually measure. One ground rule, written at the bottom: we use these terms operationally, measured in representation space. We are not proving theorems about the model's full internal state.

Attractor: where do trajectories concentrate? We measure cross-prompt distances and whether the prompt can still be recovered from a frame. Relaxation: how fast is conditioning information lost? We fit an exponential and read off a half-life. Perturbation growth: how does a small local change propagate? Twin rollouts give a finite-time growth rate. Regime transition: which settings change long-run behaviour? Same weights, different sink and window. Recurrence: does the temporal ordering carry structure? We compare against phase-randomized surrogates. And predictability: can failure be anticipated from simple statistics beforehand? That one will turn out to be a “no”, and I'll show it briefly.

Slide 10 Control: real video does not decay



Point of this slide: rule out the boring explanation before claiming anything.

Before claiming that models have attractors, we need to rule out a boring explanation: maybe all video, real or generated, drifts like this over time.

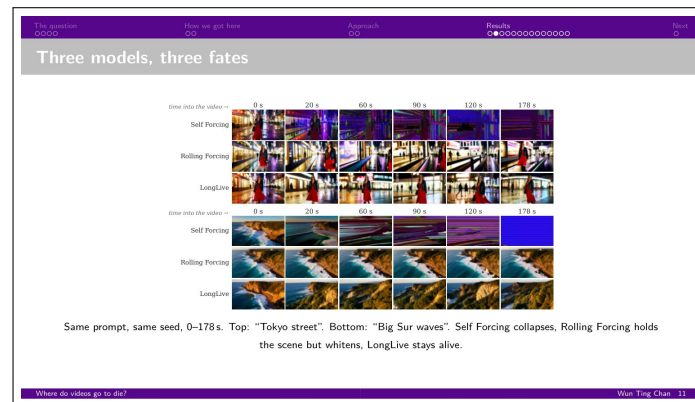
So this slide is a control. We take 128 real clips, give the models the opening and let them continue for 30 seconds, then compare against what really happened in each clip.

[left panel] The left panel measures diversity: how different the 128 videos are from each other. All curves rise in the first couple of seconds as the clips diverge from their shared start. Then they split. The black line, real video, stays flat near 0.96: real clips stay as different from each other as ever. Self Forcing, in red, falls steadily to 0.85; the 128 videos are becoming more similar to each other. Rolling Forcing in blue falls more slowly. That falling diversity is the signature of an attractor pulling everything together.

[right panel] The right panel shows one visible symptom, colour saturation. Real video, in black, stays flat. Self Forcing climbs from 0.25 to 0.55. That's the neon purple you saw on slide 3, starting within the first 30 seconds.

The dashed purple line is Self Forcing with best-of-four search, one of the safe selection methods. It's better than plain Self Forcing but still slides the same way. Searching slows the decay; it doesn't stop it.

Slide 11 Three models, three fates



Point of this slide: decay is not one phenomenon. Three models, three qualitatively different outcomes.

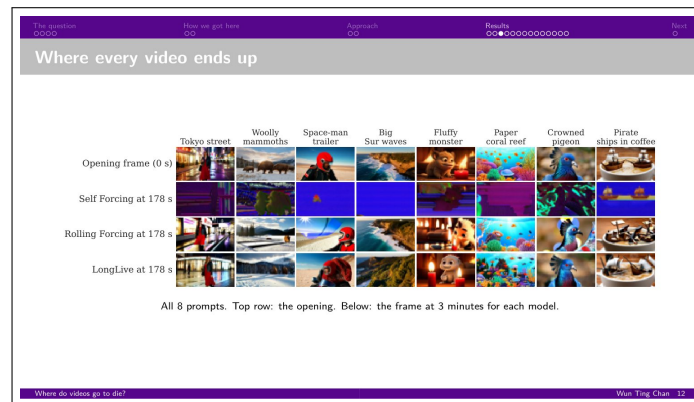
Now the three-minute runs. Same prompt and same seed, three models in each block.

[top block, Tokyo street] Self Forcing, top row, follows the path you already know: the street turns into neon streaks. Rolling Forcing, the middle row, keeps the street and the woman, but things get brighter and washed out, like glare. LongLive, the bottom row, still shows a recognisable street scene with people at three minutes.

[bottom block, Big Sur] The waves tell the same story. Self Forcing ends in a flat blue field. Rolling Forcing keeps the coastline but whitens it. LongLive is still showing waves.

So “long videos get worse” is actually three different phenomena: collapse into a shared state, slow fading in place, and staying alive. Any explanation of the decay has to account for all three.

Slide 12 Where every video ends up



Point of this slide: show the pattern holds for every prompt, not just the two we picked.

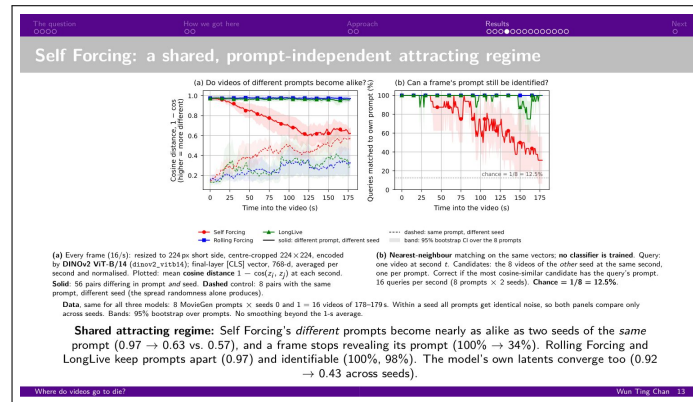
You might suspect I picked two flattering examples. So here is every prompt.

[top row] The top row is the opening frame for each of the 8 prompts. [row by row] Below it, the frame at three minutes from each model.

Read the Self Forcing row from left to right: it's essentially one image eight times, purple and blue streaks. Whatever you asked for, you end up here. The Rolling Forcing row is still eight different scenes, faded and over-bright, but you can match each one to its prompt. The LongLive row looks like eight different, living videos.

This is the visual version of the attractor claim. One model has a single global end state; the other two keep each prompt in its own basin.

Slide 13 Self Forcing: a shared, prompt-independent attracting regime



Point of this slide: turn the pictures into numbers, and make the method visible on the slide itself.

Pictures are convincing, but we want numbers. This slide makes the attractor claim quantitative with two complementary measurements, and the notes under each panel say exactly how each one is computed.

First, the representation, shared by both panels. Every frame goes through DINOv2, the ViT-B/14 version, a standard self-supervised image encoder. We take its summary vector, the [CLS] token, 768 numbers describing what's in the frame. We average the 16 frames of each second, so each video becomes one vector per second.

[panel a] Panel (a) asks: how different are videos of different prompts? At each second we compare two videos with cosine distance, one minus the cosine of the angle between their vectors. Near 1 means unrelated content; near 0 means the same content. The solid line averages over every pair of videos with a different prompt and a different seed, 56 pairs per model. The dashed line is the control: the same prompt with two different seeds, 8 pairs. It shows how far apart two videos drift through randomness alone.

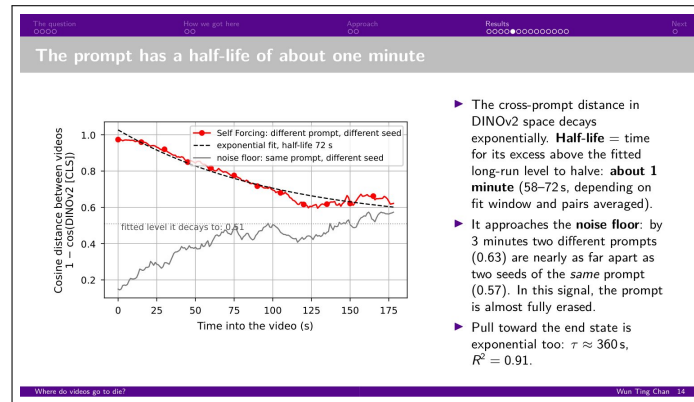
Rolling Forcing and LongLive stay flat near 0.97 for the full three minutes. Self Forcing starts there too, then falls steadily to about 0.63, close to its own dashed line at 0.57. By the end, two different prompts are nearly as alike as two runs of the same prompt.

One methodological point, because it matters. In these runs every prompt at a given seed receives identical input noise. If we compared videos within one seed, shared noise could make them look alike. So both panels only ever compare videos from different seeds, and the effect is unchanged: 0.628 across seeds against 0.636 within a seed. The bands are 95% bootstrap intervals over the eight prompts.

[panel b] Panel (b) asks the same question more intuitively: if I show you one second of video, can you tell which prompt it came from? There's no trained classifier. We take the video's vector at that second and find the most similar of the eight videos from the other seed at the same second, one per prompt. If that nearest neighbour has the same prompt, it's a hit. That's 16 queries per second, and with eight prompts chance is one in eight, 12.5%, the dotted line. Rolling Forcing stays at 100% and LongLive at 98%. Self Forcing falls from 100% to about a third.

[bottom line] Together these are what we mean by a shared, prompt-independent attracting regime, a term we use operationally. One more check supports it: the model's own latents, not just DINOv2's view of the frames, converge too, from 0.92 to 0.43 across seeds. For one model, every prompt ends up in the same place, and the prompt can no longer be read off the frame. For the other two, each prompt keeps its own place.

Slide 14 The prompt has a half-life of about one minute



Point of this slide: a single memorable number for the forgetting rate.

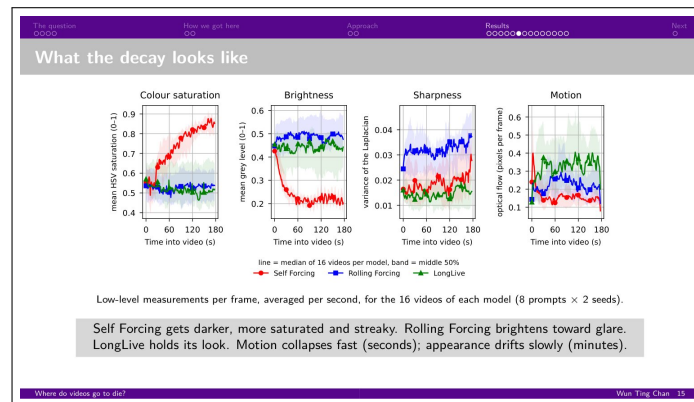
If Self Forcing forgets the prompt, how fast? This slide's job is to give that a single number.

[red line] The red line is the same Self Forcing curve as before, the distance between different prompts over time. The dashed black line is an exponential decay fitted to it, the same shape as radioactive decay. It fits well, so we can quote a half-life: about one minute. Depending on the fit window and which pairs are averaged, it comes out between 58 and 72 seconds. Precisely: the half-life is the time it takes for the excess separation between prompts, above its fitted long-run level, to halve. It describes this distance signal in DINOv2 space, not prompt information in some absolute sense.

[grey line] The grey line is the noise floor: two seeds of the same prompt. Notice where the red curve is heading: toward that floor. By three minutes, two different prompts are at 0.63 and two seeds of the same prompt at 0.57. The prompt is almost fully erased.

A second measurement, how quickly each video approaches the shared end state, is also exponential, with a time constant of about six minutes and a very clean fit ($R^2 = 0.91$). So this isn't a vague trend. It's a measurable rate.

Slide 15 What the decay looks like



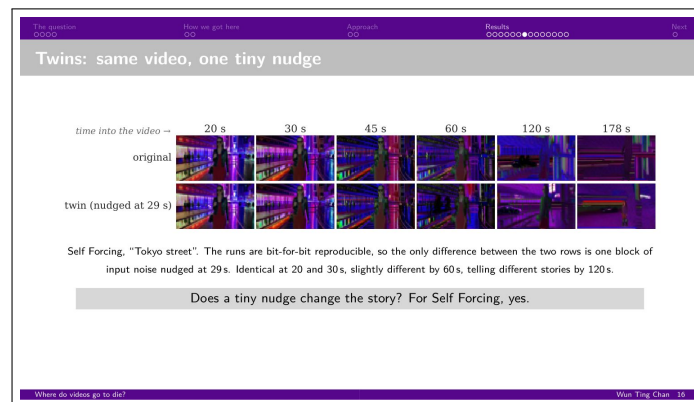
Point of this slide: connect the abstract embedding measurements back to what a viewer sees.

So far I've measured content in an abstract embedding space. This slide connects it to things a viewer would notice: colour, brightness, sharpness and motion. Each line is the median over 16 videos per model, with the band showing the spread.

[saturation] Saturation: Self Forcing, in red, climbs from about 0.55 to 0.85, oversaturated neon. The other two stay put. [brightness] Brightness: Self Forcing darkens, from 0.42 to about 0.21. Rolling Forcing, in blue, is slightly brighter than it started, which is the glare you saw. [sharpness] Sharpness: Rolling Forcing gets crisper and harsher. [motion] Motion is the interesting one. Self Forcing's motion collapses within seconds, from 0.40 to about 0.14, while its colours take minutes to drift.

So there are two clocks running: motion dies fast, appearance drifts slowly. And each model drifts in its own direction, so "decay" isn't one universal process.

Slide 16 Twins: same video, one tiny nudge

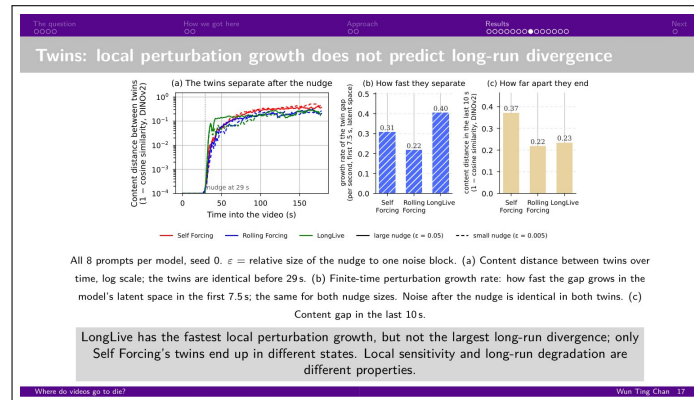


Point of this slide: set up the twin experiment visually before quantifying it.

Now a classic physics experiment: twins. These models are bit-for-bit reproducible: run the same prompt with the same seed twice and you get identical videos. So we make exactly one tiny change. At 29 seconds we nudge a single block of the random noise the model is fed, and let both copies continue.

[photo strip] Top row is the original, bottom row is the twin, both Self Forcing on the Tokyo street. At 20 and 30 seconds they're identical, as they should be. By 60 seconds there are small differences in the streaks and colours. By two minutes they're telling different stories and collapsing in different ways. For this model, a tiny nudge changes the outcome. The next slide asks whether that's true for every model.

Slide 17 Twins: local perturbation growth does not predict long-run divergence



Point of this slide: separate local sensitivity from long-run degradation; that distinction is the result.

This slide quantifies the twins for all three models, across all 8 prompts.

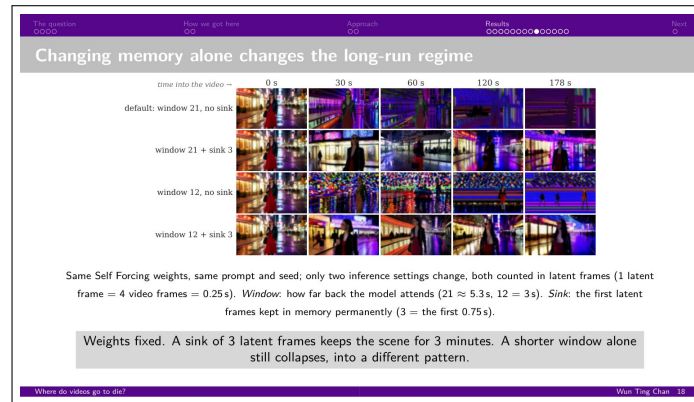
[panel a] Panel (a) tracks how far apart the twins are over time, on a log scale, using the same content distance as before. Before 29 seconds the twins are identical. That's why the lines sit on the floor of the plot: a log axis can't show zero. The noise after the nudge is identical in both twins, so the only difference is the nudge itself. Right after it, the gap for every model grows about thirtyfold within roughly ten seconds, then levels off. Solid lines are the larger nudge and dashed lines the ten-times smaller one, and they rise at the same rate. Because that early growth is roughly exponential and doesn't depend on the size of the nudge, we summarize it with a finite-time perturbation growth rate. It is Lyapunov-like, but we perturbed one block of input noise rather than the full state, so we don't claim a true Lyapunov exponent.

[panel b] Panel (b) gives that growth rate per model, measured in the model's own latent space over the first seven and a half seconds. LongLive grows fastest, at 0.40 per second, then Self Forcing at 0.31, then Rolling Forcing at 0.22.

[panel c] Panel (c) shows how far apart the twins' content ends up, over the last ten seconds. Self Forcing's twins end furthest apart, at 0.37. LongLive's end at 0.23, no further apart than Rolling Forcing's at 0.22.

So local perturbation growth does not predict long-run semantic divergence. LongLive reacts fastest to small changes but keeps telling the same story. Only Self Forcing lets a tiny nudge change where the video ends up. Local sensitivity and long-run degradation are different properties.

Slide 18 Changing memory alone changes the long-run regime



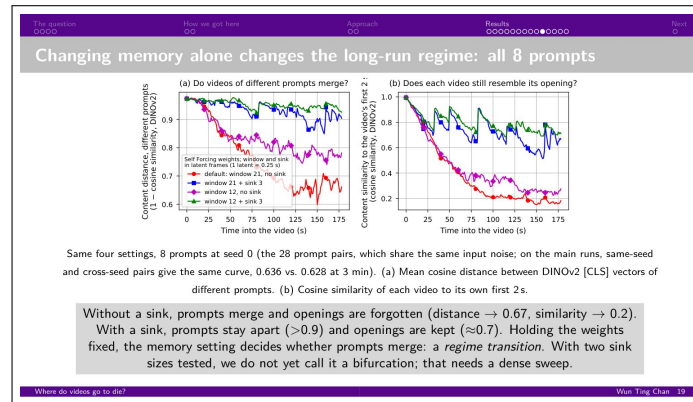
Point of this slide: the main intervention result: with weights fixed, memory design changes the long-run regime. Show it on one prompt first.

This is the intervention result I'd most like you to remember. The point is not that sinks improve video quality, which is known. The point is that a memory setting alone decides which long-run regime the system enters.

So far I've compared different models, and a sceptic could say they differ in training data, training time and a dozen other things. So here we hold the weights fixed: every row is the same Self Forcing model, same prompt, same seed. We change only two settings of how it runs, both counted in what the model calls latent frames. One latent frame is four video frames, a quarter of a second.

[rows] The top row is the default: the model looks back over 21 latent frames, about five seconds, with no sink. It collapses into streaks as usual. The second row adds a sink of 3 latent frames: the first three quarters of a second of video stay in memory permanently. Look at the end: the woman and the street are still there at three minutes. The third row shortens the window to 12 latent frames, three seconds, with no sink. It still collapses, but into a different-looking state, a bright kaleidoscope instead of streaks. The fourth row does both, and it stays anchored.

Slide 19 Changing memory alone changes the long-run regime: all 8 prompts



Point of this slide: show the single-prompt picture holds across prompts; call it a regime transition, not yet a bifurcation.

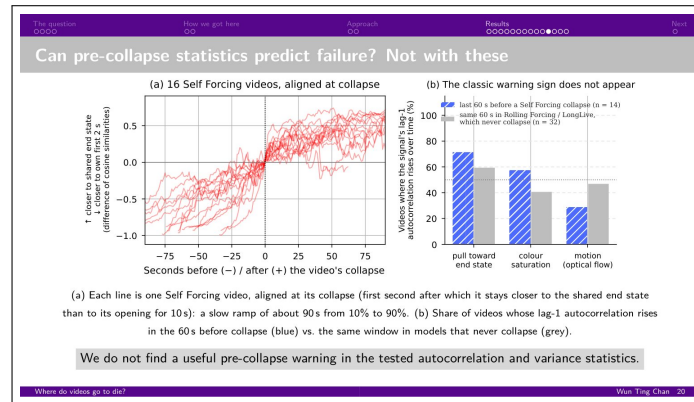
One prompt could be luck, so here are the same four settings averaged over all 8 prompts.

[panel a] Panel (a) is the distance between videos of different prompts, the measure we used to detect the shared attracting regime. These runs are seed 0 only, so the prompts share input noise; on the main runs that made no difference. Without a sink, the red default and the purple short-window lines fall: prompts merge, ending around 0.67 for the default. With a sink, the blue and green lines stay above 0.9 for the whole three minutes: prompts stay apart.

[panel b] Panel (b) asks how similar each video stays to its own first two seconds. Without a sink, similarity falls to about 0.2: the opening is forgotten. With a sink it holds around 0.7.

The point: three quarters of a second of permanent anchor switches the system from a shared attracting regime to separate, prompt-anchored ones, with the weights held fixed. That's a regime transition. I'm careful not to call it a bifurcation yet: we've tested two sink sizes, and a bifurcation needs a sweep that shows where the switch happens. That sweep is planned. It also separates the roles of the two settings: the sink controls whether the prompts merge, and the window controls what the collapse looks like.

Slide 20 Can pre-collapse statistics predict failure? Not with these



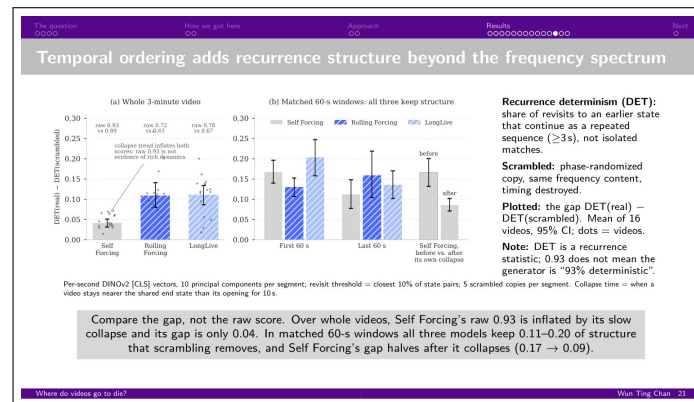
Point of this slide: a short negative result; don't dwell.

One short negative result. Panel (a) lines up all 16 Self Forcing videos at their moment of collapse, the point after which each stays closer to the shared end state than to its own opening. Each video slides in gradually: it takes about 90 seconds to go from 10% to 90% of the way.

[panel b] We asked whether simple statistics announce the collapse beforehand. A standard candidate is rising autocorrelation: each moment looking more like the last. Blue is the 60 seconds before a Self Forcing collapse, grey the same window in models that never collapse. They're about the same, and for motion it's reversed. With 14 and 32 videos, none of this is a usable warning.

So we do not find a useful pre-collapse warning in the autocorrelation and variance statistics we tested. We'll report that, but it isn't central.

Slide 21 Temporal ordering adds recurrence structure beyond the frequency spectrum



Point of this slide: use the gap, not the raw score; show what collapse does to structure.

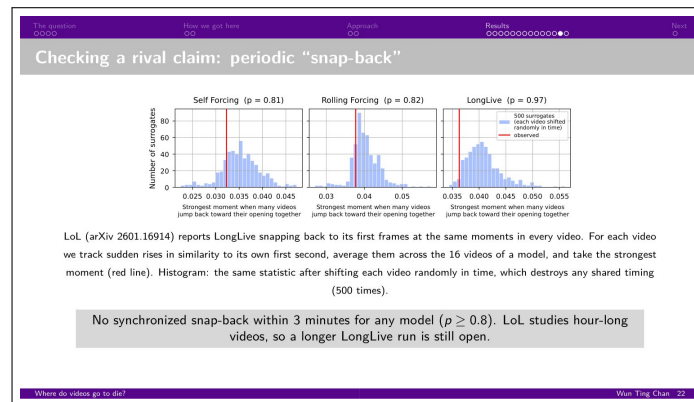
A quick but careful check of whether these trajectories have real temporal structure. The tool is recurrence analysis, and I'll define it in plain terms, as the box on the right does. Recurrence determinism asks: when a video comes back to a state it was in before, does it then repeat a whole sequence, like a pendulum retracing its path, or is it just an isolated match? The control is a scrambled copy of each video, called phase randomization. It keeps the same frequency content, the same mix of fast and slow change, but destroys the timing. So the gap between the real video and its scrambled copy is the structure that only the true ordering provides. That gap is what we plot. The raw score on its own is hard to interpret. And to be clear about the word: determinism here is a recurrence statistic, so 0.93 does not mean the generator is 93% deterministic.

[panel a] Panel (a) is the whole three-minute video. Rolling Forcing and LongLive lose about 0.11 when scrambled. Self Forcing loses only 0.04, even though its raw score is the highest at 0.93. That raw 0.93 shouldn't be read as rich dynamics. Most of it comes from the slow three-minute collapse, which the scrambled copy reproduces just as well. That's why it's greyed out.

[panel b] But a small whole-video gap doesn't mean Self Forcing has no structure, so panel (b) compares matched 60-second windows. In the first minute all three models show clear structure: Self Forcing 0.17, Rolling Forcing 0.13, LongLive 0.20. In the last minute it's 0.11 to 0.16. The rightmost pair splits each Self Forcing video at its own collapse time. Before collapse the gap is 0.17; after collapse it halves to 0.09.

[takeaway] So the message is precise. Compare the gap, not the raw score. In all three generators, temporal ordering adds recurrence structure beyond what the frequency spectrum explains. Self Forcing's high raw score is a collapse artefact. And collapse itself destroys about half of the structure. I'm deliberately not claiming that Rolling Forcing and LongLive are more structured overall, because over matched windows they aren't.

Slide 22 Checking a rival claim: periodic “snap-back”



Point of this slide: show we take competing findings seriously and test them.

Part of doing this properly is taking competing findings seriously. A recent paper, LoL, reported that LongLive periodically “snaps back” to its opening frames at the same moments in every video. That would contradict our claim that LongLive stays alive, so we tested it.

The logic: if every video snaps back at the same moments, then sudden jumps back toward the opening should line up across videos. For each video we track sudden rises in its similarity to its own first second, average those across the 16 videos of a model, and take the strongest moment. That’s the red line. Then we build a reference: shift each video randomly in time, which destroys any shared timing, recompute, and repeat 500 times. That’s the blue histogram, what chance alignment looks like.

[histograms] If snap-back were real, the red line would sit far to the right of the histogram. For all three models it sits inside it, with p-values of 0.8 and above. Individual jumps do happen, but at scattered times.

One caveat: LoL studied hour-long videos and we’ve only gone to three minutes. So the honest statement is “not within three minutes”, and a longer LongLive run is on our list.

Slide 23 Has anyone done this already?

The Journal
of Neuroscience

DOI

10.1523/JNEUROSCI.0000-0000.2024

Approach

DOI

10.1523/JNEUROSCI.0000-0000.2024

Page

2 of 2

Has anyone done this already?

About 60 targeted searches (2024 to Oct 2026), rechecked Oct 6. Long-horizon degradation is **measured widely**. What we did not find is any work that treats streaming video generators as **dynamical systems**.

Closest work	What it shows	Relation to ours
VBench-Long, Relax Forcing, In-Distribution Forcing (2610.03120) LoL (2601.16514) Deep / Rolling Sink, FreqForcing (2607.27110) Effective-rank collapse (2607.27036) Meta-ARVDM (2503.10704) Birol et al., Nat. Commun. 2024 Hintze et al., Patterns 2025	Quality, consistency, colour and motion drift over minutes Sink-collapse loops at hour scale A sink slows drift; drift as spectral-energy shift	measure symptoms, we model dynamics partial: test longer partial: we show a type switch
	Abrupt representation collapse at drift onset Theory of history forgetting Attractor-like regimes inside the denoising steps Iterated image loops collapse to ~12 attractors	reconcile with "slow ramp" frames, not the prompt other time axis concept, not video

Safe claim: existing work measures long-horizon degradation and specific failure modes; we characterize where trajectories converge, how fast prompt information is lost, how perturbations grow, and which inference settings change the long-run regime. Avoid: "nobody has measured it", "every model collapses", "sinks prevent drift" (known).

Where do videos go to die?

Wing Tan, Chan

Point of this slide: novelty, stated precisely and defensibly.

The natural question is whether this has been done. We ran about 60 targeted literature searches, covering 2024 up to this week, and rechecked them today.

Be precise about the answer. Long-horizon degradation is measured widely. VBench-Long scores quality and consistency over long videos. Papers from the last few months report colour and motion drift over minutes, spectral drift, a collapse in the model’s internal representations at drift onset, and the sink collapse I just tested. What we did not find is anyone treating these streaming generators as dynamical systems: asking where trajectories converge, how fast the prompt is lost, how perturbations grow, and which settings change the attractor.

[table] The table lists the closest work and how it relates to ours. Several sink papers already show that a sink slows drift, so our claim is the stronger one: it changes the type of attractor. The representation-collapse paper finds something abrupt where we see a slow ramp, and reconciling the two is a to-do. There is also attractor language elsewhere. Biroli and colleagues in Nature Communications describe attractor-like regimes inside the denoising steps that generate a single image. That’s a different time axis from ours, which is time within the video. And looping image models are known to collapse to about a dozen attractors.

[grey box] So here is the claim we'll make in the paper: existing work measures degradation and specific failure modes; we characterize the dynamics. What we won't claim: that nobody has measured long-video decay, that every model collapses, or that sinks prevent drift. All three would be easy to knock down.

Slide 24 Next: experiments that test the framing (planned, no results yet)

Point of this slide: be clear that these are plans; end on the one-sentence version of the paper.

Everything on this slide is planned. None of it has results yet.

[left block] On the left, generality and statistics. All three models share the Wan backbone, so the most valuable addition is a second, non-Wan model family, to show these regimes aren't Wan-specific. If we're describing distributions, two seeds isn't enough; we want 16 to 32 prompts with 4 to 8 seeds each. Then we compare the prompt-conditioned distributions directly, with distribution distances like MMD and energy distance and with prompt decoding, instead of only pairwise distances. And we'll repeat the key result in a second representation, CLIP or SigLIP, alongside DINOv2 and the model's latents.

[right block] On the right, the dynamics. A dense sink sweep from 0 to 8 tells us whether the regime switch is sharp, which is what would justify the word bifurcation. Removing Rolling Forcing's own sink tests the same mechanism in a second model. Longer runs test whether the collapsed regime is stationary, and whether a collapsed video that we perturb comes back to it, which is the classic test of attraction. State perturbations swept in size and direction would make the twin analysis rigorous. And one experiment is running right now: correcting the drift by steering each chunk away from the shared end state.

[final box] If you remember one sentence: we characterize long-horizon video generators as prompt-conditioned stochastic dynamical systems. Different generators occupy distinct long-run conditioning regimes, with measurable rates of prompt forgetting, perturbation growth and recurrence structure, and memory settings can move a model between regimes.

Thank you. Happy to take questions.